

Guía de Investigación: Grafos y su Implementación en Rust

Objetivo: Investigar de forma autónoma la estructura de datos de grafos, comprender por qué su implementación en Rust presenta un desafío único en la gestión de memoria y exponer una solución práctica utilizando herramientas estándar del lenguaje.

Fase 1: Ruta de Aprendizaje (Preguntas Guía)

Los estudiantes deben resolver este cuestionario teórico mediante su propia investigación antes de pasar al código.

1. Fundamentos de Grafos

- ¿Qué es un grafo y cuáles son sus componentes matemáticos esenciales ($G = (V, E)$)?
- ¿Cuál es la diferencia entre un grafo **dirigido** y uno **no dirigido**? ¿Qué es un grafo **ponderado**?
- Investiga y compara las dos representaciones clásicas en memoria: **Matriz de Adyacencia** y **Lista de Adyacencia**. ¿Cuándo es mejor usar una sobre la otra en términos de complejidad espacial ($O(V^2)$ vs $O(V + E)$)?

2. El "Problema del Grafo" en Rust (El núcleo de la investigación)

- En lenguajes como Java o C++, los grafos se construyen fácilmente creando nodos que apuntan directamente a otros nodos mediante punteros o referencias. ¿Por qué el **Borrow Checker** (verificador de préstamos) de Rust hace que esto sea extremadamente difícil?
- Investiga qué problemas de propiedad (*ownership*) y ciclos de memoria ocurren si intentas usar referencias normales ($\&T$) para conectar nodos.
- ¿Qué alternativas seguras ofrece Rust para solucionar esto? (Pistas a investigar: Uso de índices de vectores `Vec<usize>` o punteros inteligentes como `Rc<RefCell<T>>`).

3. Ecosistema de Rust: La Librería **petgraph**

- ¿Qué es la caja (*crate*) **petgraph** y por qué es el estándar de la industria en Rust para manejar grafos?
- Investiga cómo añade **petgraph** nodos y aristas a su estructura predefinida.

Fase 2: Aplicación Práctica (El Reto de Código)

Cada equipo o estudiante deberá escribir un programa funcional en Rust que cumpla con los siguientes requisitos mínimos:

1. **Definición del Grafo:** Crear un grafo (puede ser usando **petgraph** o mediante una lista de adyacencia propia basada en índices de vectores).

2. **Modelado:** El grafo debe representar una red real simplificada (por ejemplo: una red de transporte entre ciudades, una red social de amigos o un árbol de dependencias de software).
3. **Algoritmo:** Implementar o utilizar un algoritmo de recorrido básico:
 - **BFS** (Búsqueda en Anchura) para encontrar la ruta con menos conexiones.
 - O **DFS** (Búsqueda en Profundidad) para detectar si existen ciclos en la red.

Fase 3: Estructura de la Exposición

La presentación del alumno/equipo debe durar un máximo de **15 minutos** y estructurarse de la siguiente manera:

- **Introducción (3 min):** Explicación del concepto de grafo y el problema del mundo real que decidieron modelar.
- **El Reto de Rust (4 min):** Explicación técnica de por qué es difícil hacer grafos en Rust (demostrando comprensión de *ownership* y gestión de memoria).
- **Demostración del Código (5 min):** Mostrar el código fuente, explicar cómo estructuraron los nodos/aristas y ejecutar el programa en vivo mostrando el resultado del recorrido (BFS/DFS).
- **Conclusiones y Preguntas (3 min):** Qué aprendieron sobre el control de memoria de Rust en comparación con otros lenguajes.

Rúbrica de Evaluación (Escala de 1 a 5)

Esta rúbrica evalúa tanto el reporte de investigación como la exposición y el código entregado.

Criterio	5 - Excelente	3 - Satisfactorio	1 - Insuficiente	Puntaje
Dominio Teórico de Grafos	Demuestra comprensión perfecta de vértices, aristas, matrices/listas de adyacencia y su impacto en memoria.	Define bien los grafos, pero confunde o no explica claramente las diferencias de rendimiento entre representaciones.	No comprende la estructura de un grafo ni sus componentes básicos.	/5

Comprensión del Reto en Rust	Explica con total claridad el conflicto entre los grafos (ciclos) y el <i>Borrow Checker</i> de Rust (<i>ownership</i>).	Menciona que Rust tiene problemas con las referencias, pero no explica el trasfondo del porqué (ciclos/propiedad).	No identifica por qué Rust maneja los grafos de manera distinta a otros lenguajes.	/5
Calidad y Lógica del Código	El código en Rust es limpio, compila a la primera, no usa bloques unsafe innecesarios y ejecuta el algoritmo correctamente.	El código compila y funciona, pero la estructura es confusa o depende excesivamente de soluciones ineficientes sin justificar.	El código no compila o no realiza el recorrido (BFS/DFS) solicitado.	/5
Claridad en la Exposición	Explicación fluida, manejo excelente del tiempo, uso de analogías claras y responde con solvencia las preguntas del profesor/compañeros.	Se apoya demasiado en las diapositivas, lee texto de forma literal por momentos, pero logra transmitir la idea general.	Exposición desorganizada, excede el tiempo por mucho y demuestra inseguridad o desconocimiento al responder preguntas.	/5
Trabajo Autodidacta / Enfoque Crítico	Fue más allá de la guía: investigó optimizaciones (como localidad de caché) o comparó herramientas de manera analítica.	Se limitó estrictamente a responder las preguntas de la guía sin profundizar en las implicaciones del lenguaje.	El contenido es una copia directa de documentación externa sin análisis ni síntesis propia.	/5

Puntaje Total Máximo: 25 Puntos.